

EXPERIMENT – 4

TITLE

Build a Classification Model on Heart Disease UCI Dataset Using Ensemble Techniques with TensorFlow

Name: Sujay V

USN: 24BTRAO041

Course: MLOPS

1. AIM

To build and train a classification model using TensorFlow and Keras for predicting the presence or absence of heart disease based on different patient-related features.

The experiment demonstrates the complete machine learning workflow including dataset loading, data analysis, preprocessing, feature scaling, train-test splitting, neural network model development, model training, ensemble prediction, model evaluation, and visualization.

Three different neural network models are trained independently and their prediction probabilities are combined using an averaging ensemble technique to obtain the final classification.

2. DATASET DESCRIPTION

The dataset used for this experiment is the **Heart Disease UCI dataset**. The dataset contains **303 patient records and 13 input features**.

The dataset contains the following attributes:

- **age:** Age of the patient.
- **sex:** Gender of the patient.
- **cp:** Chest pain type.
- **trestbps:** Resting blood pressure.
- **chol:** Serum cholesterol level.
- **fbs:** Fasting blood sugar.
- **restecg:** Resting electrocardiographic results.

- **thalach:** Maximum heart rate achieved.
- **exang:** Exercise-induced angina.
- **oldpeak:** ST depression induced by exercise.
- **slope:** Slope of the peak exercise ST segment.
- **ca:** Number of major vessels.
- **thal:** Thalassemia.

The **target** variable represents the presence or absence of heart disease.

- **0 – No Disease**
- **1 – Disease**

Since the target variable contains two classes, this is considered a **binary classification problem**.

The dataset was divided into **80% training data and 20% testing data**. Feature scaling was performed using StandardScaler before training the neural networks.

3. PYTHON CODE AND OUTPUT SCREENSHOTS

The heart disease classification model was implemented using Python in Jupyter Notebook.

The required libraries including Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, and TensorFlow Keras were imported for data processing, visualization, preprocessing, neural network development, and model evaluation.

P . T . O

```
In [ ]: # Sujayv
# 24BTRA0041

# Import basic libraries for data handling
import pandas as pd
import numpy as np

# Import libraries for visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Import machine Learning Libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Import TensorFlow and Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
In [2]: # Set random seeds so results are more consistent between runs
tf.random.set_seed(42)
np.random.seed(42)
```

```
In [3]: # Load the Heart Disease UCI dataset
data = pd.read_csv(r"C:\Users\sujay\OneDrive\Desktop\Exp4_Mlops\heart.csv")

# Fix the first column name if it contains a BOM character
data.rename(columns={data.columns[0]: "age"}, inplace=True)

# Display the first 5 rows
data.head()
```

```
Out[3]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2

```
In [4]: # Check the number of rows and columns
print("Shape of dataset:", data.shape)

# Display all column names
print("Columns:", data.columns.tolist())
```

Shape of dataset: (303, 14)

Columns: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']

In [5]: `# Display information about the dataset`
`data.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         303 non-null    int64
 1   sex         303 non-null    int64
 2   cp          303 non-null    int64
 3   trestbps    303 non-null    int64
 4   chol        303 non-null    int64
 5   fbs         303 non-null    int64
 6   restecg     303 non-null    int64
 7   thalach     303 non-null    int64
 8   exang       303 non-null    int64
 9   oldpeak     303 non-null    float64
10   slope       303 non-null    int64
11   ca          303 non-null    int64
12   thal        303 non-null    int64
13   target      303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

In [6]: `# Display statistical summary of numerical columns`
`data.describe()`

Out[6]:

	age	sex	cp	trestbps	chol	fbs	restecg
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528000
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525000
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000

In [7]: `# Check the number of missing values in each column`
`print("Missing values:")`
`print(data.isnull().sum())`

Missing values:

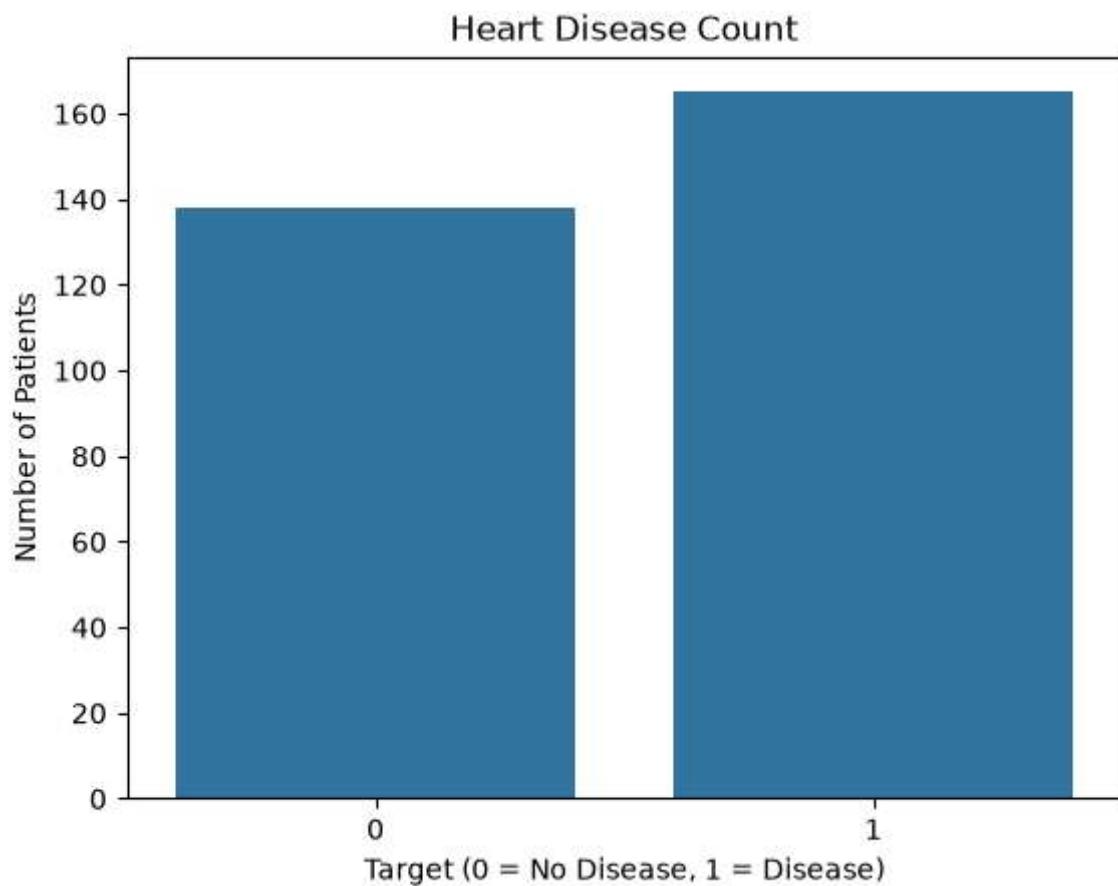
age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

dtype: int64

```
In [8]: # Check the number of patients with and without heart disease  
print(data["target"].value_counts())
```

```
target  
1    165  
0    138  
Name: count, dtype: int64
```

```
In [9]: # Plot the number of patients with and without heart disease  
sns.countplot(x="target", data=data)  
  
plt.title("Heart Disease Count")  
plt.xlabel("Target (0 = No Disease, 1 = Disease)")  
plt.ylabel("Number of Patients")  
  
plt.show()
```



```
In [10]: # X contains the input features
X = data.drop("target", axis=1)

# y contains the target variable
y = data["target"]

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

Feature shape: (303, 13)
Target shape: (303,)

```
In [11]: # Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)
```

Training data size: (242, 13)
Testing data size: (61, 13)

```
In [12]: # Create a StandardScaler object
scaler = StandardScaler()

# Fit the scaler only on training data
X_train = scaler.fit_transform(X_train)

# Apply the same scaling to test data
X_test = scaler.transform(X_test)

print("Feature scaling completed.")
```

Feature scaling completed.

```
In [13]: # Function to build a neural network model
def build_model(units1, units2, seed):

    # Set the random seed for this model
    tf.random.set_seed(seed)

    # Create the neural network
    model = keras.Sequential([
        layers.Input(shape=(X_train.shape[1],)),
        layers.Dense(units1, activation="relu"),
        layers.Dense(units2, activation="relu"),
        layers.Dense(1, activation="sigmoid")
    ])

    # Compile the model
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
```

```
return model
```

```
In [14]: # Define different configurations for the three neural networks
model_settings = [
    {"units1": 16, "units2": 8, "seed": 1},
    {"units1": 32, "units2": 16, "seed": 2},
    {"units1": 8, "units2": 8, "seed": 3}
]

# Create an empty list to store the trained models
models = []
```

```
In [15]: # Train each neural network separately
for i, setting in enumerate(model_settings):

    print("Training Model", i + 1)

    # Build the model
    model = build_model(
        setting["units1"],
        setting["units2"],
        setting["seed"]
    )

    # Train the model
    model.fit(
        X_train,
        y_train,
        epochs=50,
        batch_size=16,
        verbose=0
    )

    # Store the trained model
    models.append(model)

print("All models trained successfully!")
```

Training Model 1

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Training Model 2

Training Model 3

All models trained successfully!

```
In [16]: # Create a list to store individual model accuracies
individual_accuracies = []

# Test every model separately
for i, model in enumerate(models):

    # Predict probabilities
    pred_prob = model.predict(X_test, verbose=0)

    # Convert probabilities into classes
    pred_label = (pred_prob > 0.5).astype(int).flatten()

    # Calculate accuracy
```

```

acc = accuracy_score(y_test, pred_label)

# Store accuracy
individual_accuracies.append(acc)

print("Model", i + 1, "Accuracy:", round(acc * 100, 2), "%")

```

Model 1 Accuracy: 80.33 %

Model 2 Accuracy: 78.69 %

WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000002BD0CD27920> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000002BD0CD27920> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

Model 3 Accuracy: 81.97 %

```

In [17]: # Create a list to store predictions from all models
all_predictions = []

# Generate predictions from every trained model
for model in models:

    # Get predicted probabilities
    pred_prob = model.predict(X_test, verbose=0)

    # Store predictions
    all_predictions.append(pred_prob.flatten())

# Convert predictions into a NumPy array
all_predictions = np.array(all_predictions)

print("Prediction shape:", all_predictions.shape)

```

Prediction shape: (3, 61)

```

In [18]: # Calculate the average probability from all three models
ensemble_prob = np.mean(all_predictions, axis=0)

# Convert the average probability into class 0 or 1
ensemble_pred = (ensemble_prob > 0.5).astype(int)

print("Ensemble prediction completed.")

```

Ensemble prediction completed.

```

In [19]: # Calculate the accuracy of the ensemble model
ensemble_acc = accuracy_score(
    y_test,

```



```

    ensemble_pred
)

print(
    "Ensemble Model Accuracy:",
    round(ensemble_acc * 100, 2),
    "%"
)

```

Ensemble Model Accuracy: 80.33 %

```

In [20]: # Store the accuracy of all models
results = {}

for i, acc in enumerate(individual_accuracies):
    results["Model " + str(i + 1)] = acc

# Add ensemble accuracy
results["Ensemble (Average)"] = ensemble_acc

# Display accuracy comparison
print("Accuracy Comparison:")

for name, acc in results.items():
    print(name, ":", round(acc * 100, 2), "%")

```

Accuracy Comparison:
 Model 1 : 80.33 %
 Model 2 : 78.69 %
 Model 3 : 81.97 %
 Ensemble (Average) : 80.33 %

```

In [21]: # Plot accuracy comparison of individual models and ensemble
plt.figure(figsize=(8, 5))

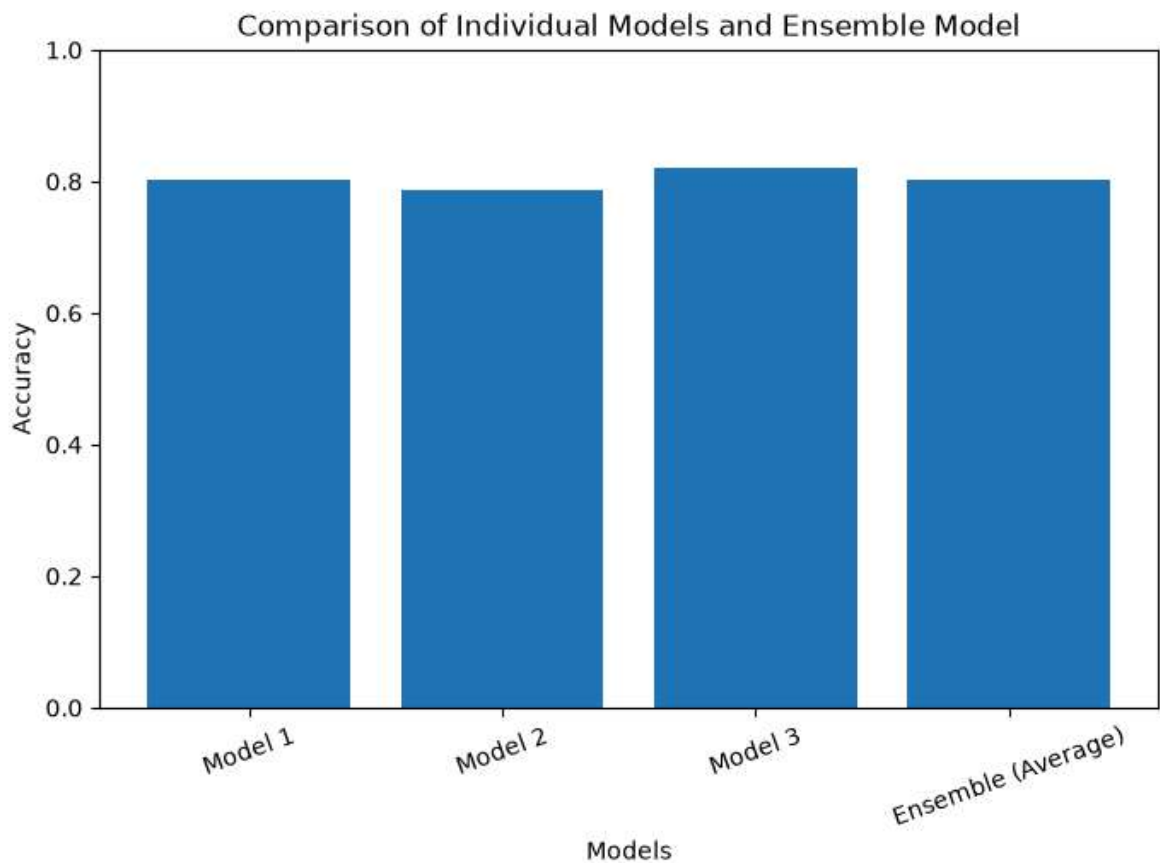
plt.bar(
    results.keys(),
    results.values()
)

plt.ylabel("Accuracy")
plt.xlabel("Models")
plt.title("Comparison of Individual Models and Ensemble Model")

plt.ylim(0, 1)
plt.xticks(rotation=20)

plt.show()

```



```
In [22]: # Calculate the confusion matrix
cm = confusion_matrix(
    y_test,
    ensemble_pred
)

# Display the confusion matrix
print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:

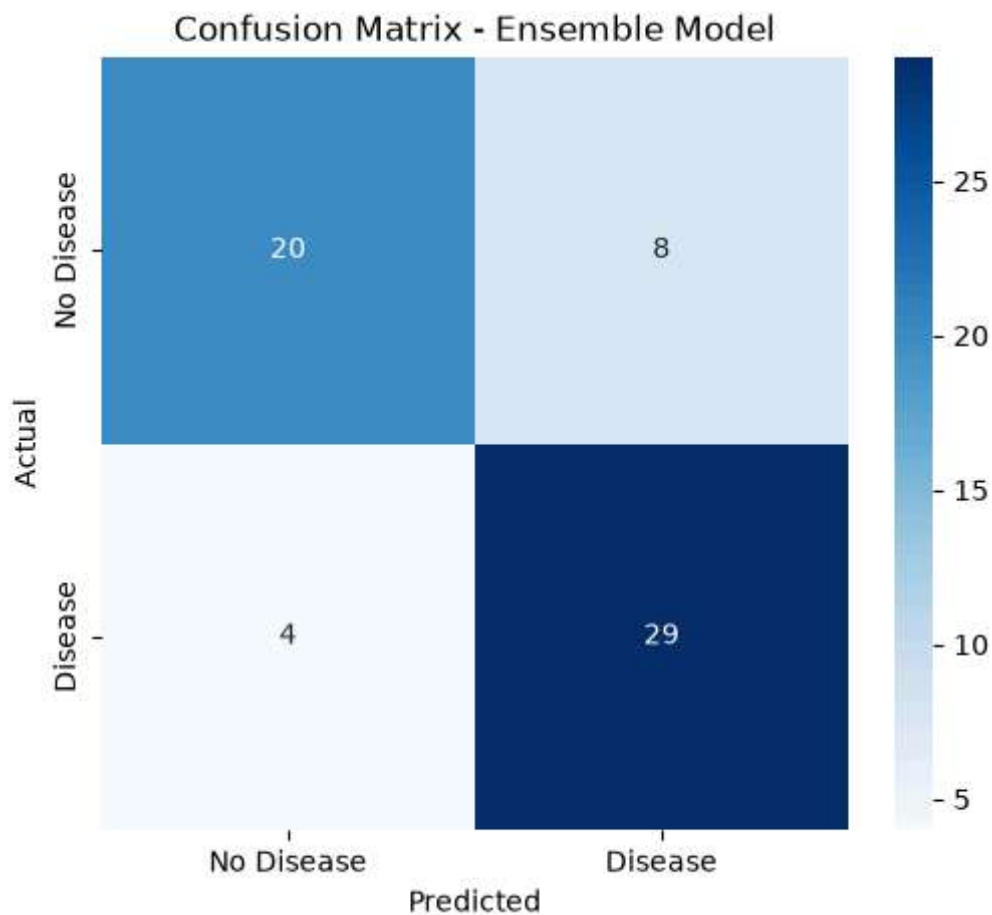
```
[[20  8]
 [ 4 29]]
```

```
In [23]: # Plot the confusion matrix
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Disease", "Disease"],
    yticklabels=["No Disease", "Disease"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Ensemble Model")

plt.show()
```



```
In [24]: # Generate the classification report
print("Classification Report:")

print(
    classification_report(
        y_test,
        ensemble_pred
    )
)
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.83         0.71         0.77         28
     1       0.78         0.88         0.83         33

 accuracy          0.80         0.80         0.80         61
 macro avg         0.81         0.80         0.80         61
 weighted avg      0.81         0.80         0.80         61
```

```
In [25]: # Display the final accuracy of all models
print("Final Results:")

for name, acc in results.items():
    print(
        name,
        ":",
        round(acc * 100, 2),
        "%"
    )
```

```
print("\nExperiment completed successfully!")
```

Final Results:

Model 1 : 80.33 %

Model 2 : 78.69 %

Model 3 : 81.97 %

Ensemble (Average) : 80.33 %

Experiment completed successfully!

4. MODEL EVALUATION RESULTS

The trained neural network models were evaluated using classification accuracy, confusion matrix, precision, recall, and F1-score.

The dataset contained **303 records**, with 80% used for training and 20% used for testing.

CLASSIFICATION PERFORMANCE

Model	Accuracy
Model 1	80.33%
Model 2	78.69%
Model 3	81.97%
Ensemble	80.33%

The best individual model was **Model 3**, which achieved an accuracy of **81.97%**.

The averaging ensemble achieved an accuracy of **80.33%**.

CONFUSION MATRIX

[[20 8]

[4 29]]

The ensemble model correctly classified **20 patients without heart disease** and **29 patients with heart disease**.

It incorrectly classified **8 patients without heart disease as having disease** and **4 patients with heart disease as having no disease**.

CLASSIFICATION PERFORMANCE

For the **No Disease** class, the model achieved:

- Precision: **0.83**
- Recall: **0.71**
- F1-Score: **0.77**

For the **Disease** class, the model achieved:

- Precision: **0.78**
- Recall: **0.88**

- F1-Score: **0.83**

The disease class achieved a recall of **0.88**, indicating that the model correctly identified a large proportion of the patients belonging to the disease class.

RESULT ANALYSIS

The three TensorFlow neural network models produced different classification accuracies.

- **Model 1:** 80.33%
- **Model 2:** 78.69%
- **Model 3:** 81.97%
- **Ensemble:** 80.33%

Among the individual models, **Model 3 achieved the highest accuracy of 81.97%.**

The averaging ensemble achieved an accuracy of **80.33%**. Therefore, in this experiment, the ensemble model did not outperform the best individual model. However, it successfully combined the prediction probabilities of all three neural networks.

The confusion matrix showed that the ensemble model correctly predicted 20 no-disease cases and 29 disease cases. It incorrectly predicted 8 no-disease cases as disease and 4 disease cases as no disease.

The classification report showed an F1-score of **0.77** for the no-disease class and **0.83** for the disease class.

5. GITHUB REPOSITORY LINK

The complete implementation of the Heart Disease Classification model is available on GitHub.

GitHub Repository:

https://github.com/sujayv26/Heart_disease_classification_tensorflow.git

6. CONCLUSION

The TensorFlow-based heart disease classification model was successfully implemented using the Heart Disease UCI dataset.

The experiment demonstrated the complete machine learning workflow, including dataset loading, data analysis, preprocessing, feature scaling, train-test splitting, neural network model creation, model training, ensemble prediction, evaluation, and visualization.

Three different neural network models were created using TensorFlow and Keras with different hidden-layer configurations and random seeds. The models were trained using the Adam optimizer and binary cross-entropy loss function.

The individual models achieved accuracies of **80.33%**, **78.69%**, and **81.97%**, while the averaging ensemble achieved an accuracy of **80.33%**.

Model 3 achieved the highest individual accuracy of **81.97%**. The ensemble model was further evaluated using a confusion matrix and classification report and achieved an F1-score of **0.77 for the no-disease class** and **0.83 for the disease class**.

Thus, the experiment demonstrates the practical application of TensorFlow, Keras, neural networks, and ensemble learning techniques for solving a real-world binary classification problem such as heart disease prediction.